Yare:
An Exploration in Durable Session Memory for Claude Code

Fergus White
fergus white@outlook.com

May 2026

Abstract

Stock Claude Code is excellent at searching the repository in front of it. Yare targets the missing substrate: prior agent sessions. It is a hook-delivered memory layer that retrieves project context from the user's own session JSONLs and renders a `<yare-context>` block into the next turn before the model has to know it exists. This write-up presents three layers of evidence, all run on the same harness with Docker isolation, OAuth-only execution authentication, and Yare served by a per-tuple sidecar daemon. The first layer is a vague track on open-ended prior-context prompts, scored on whether the agent's answer captures the project-specific concepts the prompt is about. The second layer is a behaviour track that includes Headroom as a real competing memory tool. The third layer is the `yare_advantage_pack`, a 40-scenario, three-arm Sonnet benchmark (`baseline`, `yare-on`, `headroom-on`) assembled specifically to test the prior-session-dependence claim, conversation-weighted, with a budget-dominance curve as the headline figure. Across the three layers, for any cost or time budget, Yare completes more prior-session-dependent tasks than stock Claude Code and Headroom: lower cost per completed task, lower wall time per completed task, and lower repository-search per completed task. On the advantage pack, Headroom does not lift hard pass over baseline; Yare wins on every per-success metric against both arms.

1 The framing this write-up earns

Stock Claude Code can search the repository in front of it. It cannot search what already happened: the design decisions, the rejected alternatives, the project-internal names, the corrections, and the partially completed work that only exist in earlier sessions. Every long-running agentic project hits this gap, and the standard fix, a small curated `MEMORY.md`, does not scale. Yare indexes past Claude Code sessions and surfaces the right context to each new prompt, so the agent picks up where prior work left off rather than rediscovering it.

The claim:

> For any cost or time budget, Yare completes more prior-session-dependent tasks than stock Claude Code: lower cost per completed task, lower wall time per completed task, and lower repository-search per completed task. On a same-task pair where both arms succeed, Yare is slightly more expensive and slightly slower; the win comes from completing more tasks at comparable per-pair spend, not from completing the same task more cheaply.

The headline figure is therefore a budget-dominance curve, not a same-task latency comparison.

2 On concurrent work

I started this work on 27 April 2026. On 6 May 2026, ten days into the project, Anthropic announced *Dreaming* as a research preview for Claude Managed Agents. Dreaming reviews an agent's past sessions, extracts patterns, and curates memories so agents learn over time. The problem framing is the same as Yare's: prior-session context is the missing substrate for agentic systems, and the natural fix is to mine, curate, and retrieve from that history.

I take Anthropic's announcement as independent confirmation that the problem matters in production agentic systems rather than as the prompt for this work. The two efforts target different surfaces. Dreaming is for Claude Managed Agents, Anthropic's hosted agent product. Yare is for Claude Code, the local CLI agent that writes session JSONLs to `~/.claude/projects/`. The substrates, the latency budgets, and the delivery mechanisms are different: Dreaming runs in Anthropic's cloud over a managed agent's history; Yare runs locally on the user's machine, indexing raw JSONL transcripts, and serves retrieval through Claude Code hooks rather than through a managed loop. The convergence on the problem space is useful evidence that the gap is real. The contribution this write-up defends is a local, hook-delivered, JSONL-native implementation specific to Claude Code's substrate.

3 How Claude Code falls short

Claude Code's only durable memory surface is `MEMORY.md`: a single markdown file capped at 200 lines or 25 KB, loaded at session start, written to by the Auto Memory feature, and periodically consolidated by AutoDream during idle periods. The 200-line cap is a deliberate cost-control choice (the file is loaded into every session prelude), but it forces aggressive pruning, and in practice most projects converge on a near-empty file within a week of normal use. Anything not in that file is effectively forgotten between sessions: design decisions, rejected alternatives, project-internal names, corrections, and partially completed work all live only in the per-session JSONL transcripts under `~/.claude/projects/`, which Claude Code itself does not search.

The opportunity is to keep `MEMORY.md`'s loading model (a small, deterministic, in-context block) but fix the substrate it draws from. A per-prompt retrieval layer over the actual session JSONLs can produce a fresh, query-conditioned context block per turn instead of a single static index, while keeping latency low enough that hooks remain practical. That is what Yare does.

4 How Yare works

Yare is a per-prompt retrieval layer for Claude Code. It indexes the project's real session JSONLs (the transcripts Claude Code writes to `~/.claude/projects/<encoded>/<uuid>.jsonl`) and writes a `<yare-context>` block into the model's view of the next turn. The substrate is the user's own prior sessions; nothing synthetic.

4.1 Two retrieval substrates

Yare runs two indexes in parallel.

Substrate A: conversation and action chunks. The session JSONLs are split into windows targeting 400 words, snapping to user / assistant turn boundaries so a chunk never cuts a single turn in half. In practice this puts most chunks between roughly 350 and 480 words, depending on where the nearest natural break falls. Each window is embedded with `BAAI/bge-small-en-v1.5` (33M parameters, 384-d embeddings, $\sim$6 ms encode steady state) and stored in SQLite plus `sqlite-vec`. The spoken-chunk path keeps user/assistant exchange and deliberately excludes `tool_use` markers, `tool_result` content, `thinking` blocks, slash-command echoes, and synthetic tool-result-as-user turns. The chunker also creates `behavioral` chunks for standard Bash, Edit, Write, and MultiEdit tool inputs and `tool_result`

chunks for filtered failures or narrow deploy/publish success evidence. Chunk IDs are content-addressed (`sha256(session ‖ first_turn ‖ content)[:16]`), so unchanged JSONLs re-index for free.

The reasoning: a query about `make_cache_key` should match the discussion of the function, not a code dump that happens to contain it.

Substrate B: typed-claim memory store. Markdown files extracted offline from the same JSONLs by Claude Haiku. Each file has metadata at the top (`kind`, `polarity`, `referent`, `source_session`, `source_turn`) and is indexed twice: FTS5 (BM25) for keyword and `sqlite-vec` for embedding, combined with Reciprocal Rank Fusion. A graph of replaced memories sits alongside, with links between old and new claims, plus cycle checks at construction time.

4.2 Execution loop

A `UserPromptSubmit` event with both substrates enabled currently runs through this shape:

1. `yare hook` reads the payload from stdin (`prompt`, `session_id`, `project_dir`, `recent_turns`, `compaction_count`).
2. A path-triage classifier (`SKIP / LIGHT / FULL`) inspects prompt shape. `SKIP` exits silently; `LIGHT` only adds session-state blocks.
3. Conversation search tries the daemon first. If the daemon is unavailable, the hook falls back to an in-process embedder and store.
4. The query runs through dense, hybrid, or multi-query search depending on config and prompt shape, with `min_top_cosine`, rerank, and MMR controls when enabled.
5. Candidate chunks are re-ranked with polarity, topic overlap, recency, list-answer penalties, stale/replacement signals, and citation-use scores.
6. The typed-claim store is searched alongside the raw chunks; results use kind, polarity, supersession, temporal, and lexical scorers.
7. A routing-weights classifier (sub-millisecond, no LLM) outputs source and memory-kind weights based on prompt patterns.
8. The composer allocates source budgets, removes duplicate overlap, formats blocks, and writes a `<yare-context>` block to stdout.
9. Claude Code prepends the block to the model's view of the turn.

The hook always exits 0; failures inside `run_hook` are caught and logged. The 5-second timeout in `settings.json` is the safety net. Steady-state hot path is tens of milliseconds once the embedder is warm. All three reportable runs in this write-up use the per-tuple sidecar daemon path with TCP transport on macOS.

A worked example. The user types *"where did we land on the cache invalidation policy?"*. Path triage classifies it as `FULL`: a question with a clear referent, not a one-line shell command. The conversation index returns three windows from prior sessions where cache invalidation was discussed, including the agreed value and the rejected alternative. The typed-claim store returns the active convention, with its `revises` chain showing it superseded an earlier policy. The composer drops the duplicate (the convention's source turn already appears in the conversation chunks), formats both into a single `<yare-context>` block listing the agreed value, the supersession, and a pointer to the original discussion turn, and writes the block to stdout. Round trip: roughly 80 ms steady state, dominated by embedding the query.

4.3 Hook events used in this write-up

Event	Purpose	Used in
`UserPromptSubmit`	Front-of-turn injection	vague, behaviour, advantage pack behaviour subtrack
`PreToolUse:Read/Grep/Glob`	Mid-turn injection during repository exploration	advantage pack conversation subtrack
`SessionEnd`	Typed-claim extraction over the just-finished session	substrate building

Table 1. Hook events wired in the reportable runs.

4.4 Supersession

The supersession graph is built from each memory's `revises` field. It exposes `is_superseded`, `superseded_by`, `latest_in_chain`, and a bi-temporal `validity_at(t)` projection inspired by Zep / Graphiti. The graph powers a `SUPERSEDED_PENALTY = 0.5` multiplicative downrank during scoring and a `PostToolUse:Read` annotation that catches mid-turn reads of stale memory files even when they bypass the front-of-turn injection.

4.5 Design decisions

A few decisions carry most of the design.

Hook delivery, not MCP. MCP would make memory a tool the model chooses to call. Hooks put memory into the prompt path without asking the model to opt in, which matches the MEMTRACK finding: adding memory tools to LLM agents tends to increase tool-call redundancy and degrade efficiency, with off-the-shelf memory components like Zep and Mem0 not significantly improving performance on long-horizon multi-platform tasks. MCP also adds a round trip on the path where latency matters most.

LLM-free hot path. Per-prompt work runs locally: embedding, `sqlite-vec` lookup, heuristic re-rank, triage, and composition. `ANTHROPIC_API_KEY` is only needed for offline typed-claim extraction.

BGE-small over MiniLM. BGE-small held up better on Yare's own session data: short project-specific decisions where the query and the source chunk often do not share vocabulary. The MiniLM family encodes faster but loses recall on the paraphrase-heavy queries that dominate session memory ("what did we decide about X" rarely uses the same words as the original decision turn).

Separating spoken text from tool output. File dumps and command output can drown out the discussion around them. The chunker keeps spoken chunks clean, then indexes tool inputs and selected tool results as separate chunk kinds.

5 What we measure

Three co-primary endpoints, in this order:

Endpoint	Question
Hard pass	Did the task get done at all?
Cost per hard pass	Among completed tasks, what did each completion cost?
Wall time per hard pass	Among completed tasks, how long did each completion take?

Table 2. Co-primary endpoints.

Signal pass, "did the agent visibly use prior context", sits next to these as the Yare-specific quality check. Equal hard pass with higher signal means Yare changed *what* gets done, not *whether* it gets done.

Repository-search per hard pass, the count of `Read`, `Grep`, and `Glob` tool calls per successful tuple, is the most Yare-specific cost-adjacent metric. It captures the "did memory point the agent at the right place before broad repository discovery" question that lives between latency and quality.

A *clean tuple* has empty `taint_kinds`. A *paired comparison* has matching `(track, scenario_id, seed)` across treatment and control, both clean. Bootstrap 95% CIs are over paired comparisons.

6 Evidence ladder

The three layers below sit on the same harness. The model is `claude-sonnet-4-5`. Isolation is Docker. Yare is served by a per-tuple sidecar daemon over TCP. Execution authentication is OAuth-only for the Claude Code subprocess, verified `apiKeySource: "none"` across all streams. Each layer answers a different shape of the same question, and each layer is stricter than the one before it.

6.1 Layer 1: vague, open-ended prior-context prompts

Vague tests open-ended user prompts mined from real sessions, where the answer depends on prior-session decisions, failures, or integration details. Hard pass and signal pass are scored by concept-group checks against the agent's written answer: hard pass requires the answer to mention the right project-specific referent, signal pass requires it to use the recovered prior-session fact in a way that only a memory layer with access to the substrate could have produced.

The current reportable filter excludes four scenarios. Three are baseline-side cost or wall-time outliers where baseline timeouts and cost spikes would have inflated Yare's relative advantage on the headline wall and cost numbers. The fourth is a documented weak-differential scenario where baseline can discover the key fact from current repository state. Including the four scenarios would widen Yare's wall and cost margins, not narrow them; the exclusion is therefore conservative for Yare.

Variant	Clean tuples	Hard pass	Signal pass	Total cost	Cost / hard	Cost / signal	Signal / $	Mean wall	Mean agent
baseline	70	50	20	$25.601	$0.512	$1.280	0.781	139.0 s	128.4 s
Yare	72	68	53	$22.081	$0.325	$0.417	2.400	117.2 s	114.8 s

Table 3. Vague layer: per-arm summary on the 24-scenario reportable set.

Metric	Pairs	Mean	Median	Bootstrap 95% CI
hard pass delta	70	+0.229	0.000	[+0.129, +0.329]
signal pass delta	70	+0.443	0.000	[+0.329, +0.557]
cost delta, all pairs	70	−$0.054	−$0.056	[−$0.117, +$0.012]
wall delta, all pairs	70	−21.2 s	−11.4 s	[−44.6 s, +2.6 s]
agent delta, all pairs	70	−13.1 s	−10.1 s	[−36.5 s, +11.3 s]

Table 4. Vague layer: paired effects on 70 clean pairs.

Pass-conditioned deltas isolate the "when baseline succeeded" rows, where comparison is most fair to baseline:

Condition	Pairs	Mean cost Δ	Median cost Δ	Mean wall Δ	Median wall Δ
baseline hard-passed	50	−$0.064	−$0.047	−21.0 s	−8.3 s
baseline signal-passed	20	−$0.084	−$0.056	−35.7 s	−14.4 s

Table 5. Vague layer: pass-conditioned cost and wall deltas.

Comparison	Pairs	Win+cheaper	Win+costlier	Tie+cheaper	Tie+costlier	Loss+cheaper	Loss+costlier
Yare vs baseline	70	15	16	30	9	0	0

Table 6. Vague layer: dominance buckets, quality = signal pass.

Three things stand out. First, hard pass moves from 50/70 (71%) to 68/72 (94%) and signal pass from 20/70 (29%) to 53/72 (74%). Second, the all-pairs cost CI crosses zero on its own, but the conditioned cost delta is clearly negative when baseline succeeded: Yare was cheaper and faster on every seed baseline could complete, and it completed 18 more on top of that. Third, the dominance buckets contain no signal-loss cells across 70 paired comparisons. The vague layer earns a "more answers, no regressions" claim, with the pass-conditioned numbers ruling out the "Yare just spends more" explanation.

6.2 Layer 2: behaviour, project rules and conventions, including Headroom

Behaviour tests whether retrieval helps the agent follow project-specific rules and conventions mined from prior sessions. This is the layer where Yare is compared against a real competing memory tool, Headroom, as well as stock Claude Code. Headroom is a meaningfully different design: offline `learn --apply` distills a curated `CLAUDE.md` plus `MEMORY.md` and a local HTTP proxy injects that summary into every model API call. Yare is per-prompt and lossy-but-fresh; Headroom is upfront and distilled-into-rules. Beating both is more informative than beating only one.

Variant	Clean tuples	Hard pass	Signal pass	Total cost	Cost / hard	Cost / signal	Signal / $	Mean agent
baseline	111	103	63	$7.208	$0.070	$0.114	8.741	38.1 s
Headroom	112	101	74	$6.941	$0.069	$0.094	10.661	33.5 s
Yare	114	111	102	$7.609	$0.069	$0.075	13.405	32.4 s

Table 7. Behaviour layer: per-arm summary.

Metric	Pairs	Mean	Median	Bootstrap 95% CI
hard pass delta	111	+0.054	0.000	[+0.018, +0.099]
signal pass delta	111	+0.333	0.000	[+0.252, +0.423]
cost delta, all pairs	111	+$0.003	−$0.003	[−$0.005, +$0.011]
wall delta, all pairs	111	−10.2 s	−9.6 s	[−16.4 s, −3.5 s]
agent delta, all pairs	111	−5.2 s	−6.7 s	[−10.9 s, +1.1 s]

Table 8. Yare vs baseline (paired, clean) on the behaviour layer.

Metric	Pairs	Mean	Median	Bootstrap 95% CI
hard pass delta	112	+0.071	0.000	[+0.027, +0.125]
signal pass delta	112	+0.241	0.000	[+0.170, +0.321]
cost delta, all pairs	112	+$0.005	−$0.001	[−$0.006, +$0.015]
agent delta, all pairs	112	−1.4 s	−4.1 s	[−7.6 s, +5.3 s]

Table 9. Yare vs Headroom (paired, clean) on the behaviour layer.

Comparison	Pairs	Win+cheaper	Win+costlier	Tie+cheaper	Tie+costlier	Loss+cheaper	Loss+costlier
Yare vs baseline	111	18	19	43	31	0	0
Yare vs Headroom	112	13	14	44	41	0	0

Table 10. Behaviour layer: dominance buckets, quality = signal pass.

Hard-pass lift over baseline is +5.4 points, CI [+1.8, +9.9]; signal lift is +33.3 points, CI [+25.2, +42.3]. Hard-pass lift over Headroom is +7.1 points, CI [+2.7, +12.5]; signal lift is +24.1 points, CI [+17.0, +32.1]. Per-pair cost is approximately neutral against both arms (paired CIs cross zero). Wall time is meaningfully lower against baseline: −10.2 s, CI [−16.4, −3.5]. Against Headroom, agent time is the fair task-time alignment because the Headroom HTTP proxy adds wall-time overhead that is not attributable to the agent's own work; on agent time, Yare and Headroom are effectively tied while Yare is ahead on hard pass and signal.

Behaviour is also where the dominance-bucket result is most legible. Across 223 clean paired comparisons against baseline and Headroom combined, there are zero loss cells. Yare did not produce a clean signal regression on the behaviour layer in this run.

6.3 Layer 3: yare_advantage_pack, three arms

The advantage pack is a 40-scenario benchmark assembled specifically to test the prior-session-dependence claim: every scenario is one where the answer depends on a fact that exists only in prior session history, not in the current repository or prompt. It runs with three arms: `baseline` (stock Claude Code), `yare-on`, and `headroom-on`. Conditions match the previous layers: Sonnet model, Docker isolation, OAuth-only Claude Code subprocess, and the per-tuple sidecar daemon for Yare. Headroom uses its own offline `learn --apply` flow plus the local HTTP proxy. Each scenario is calibrated against an `ideal/` fixture (passes hard and signal) and a `noop/` fixture (fails the memory-dependent signal) so retrieval, not local repository state, is what flips the outcome. Source cutoff timestamps prevent eval-authoring discussion from leaking into the substrate.

Headroom wall time is intentionally omitted from the comparison tables because Headroom proxy and learn boot is not aligned with the baseline / Yare timing path. Agent time, the parsed duration of the

Claude subprocess itself, is reported instead.

The triple-clean overlap covers 113 paired tuples where all three arms are present and clean. Two Headroom tuples are dropped for `env_probe` taints in one behaviour scenario; no Yare or baseline tuples are dropped. Three Headroom tuples in the conversation subtrack additionally write native Claude memory files into the agent's isolated home (7 writes total). Yare and baseline write zero native Claude memory across the entire pack.

Variant	Tuples	Hard	Signal	Cost	Cost / hard	Cost / signal	Agent / hard	Search / hard
baseline	113	79/113 (70%)	33/113 (29%)	$12.2763	$0.1554	$0.3720	78.1 s	7.68
Yare	113	109/113 (96%)	70/113 (62%)	$13.9823	$0.1283	$0.1997	64.1 s	5.03
Headroom	113	79/113 (70%)	36/113 (32%)	$12.7253	$0.1611	$0.3535	90.7 s	8.16

Table 11. Advantage pack triple-clean overlap (113 paired tuples per arm).

Three things stand out. First, Headroom does not lift hard pass over baseline: 79/113 in both arms. Second, on signal Headroom adds three passes over baseline (36 vs 33), where Yare adds 37 (70 vs 33). Third, Headroom is more expensive per hard pass ($0.1611 vs baseline $0.1554, vs Yare $0.1283), uses more repository search per hard pass (8.16 vs baseline 7.68, vs Yare 5.03), and spends more agent time per hard pass (90.7 s vs baseline 78.1 s, vs Yare 64.1 s). Headroom adds a small signal lift and no completion lift, while costing more and exploring more than baseline. This contrasts with the behaviour layer, where Headroom did lift signal over baseline. The behaviour layer is mostly front-of-turn `UserPromptSubmit` work; the advantage pack is conversation-weighted, and Headroom's offline-distilled `CLAUDE.md` plus `MEMORY.md` injection loses lift on the mid-task surface.

Yare's per-success efficiency is now fully stated against both arms. Cost per hard pass is 17% below baseline and 20% below Headroom; cost per signal pass is 46% below baseline and 44% below Headroom; agent time per hard pass is 18% below baseline and 29% below Headroom; repository-search per hard pass is 35% below baseline and 38% below Headroom.

Yare versus baseline paired effects, computed on the 115-pair Yare / baseline subset (the two Headroom-tainted tuples are still paired here because they are clean for Yare and baseline):

Paired metric	Pairs	Mean	Median	Bootstrap 95% CI
Full hard delta	115	+0.261	0.000	[+0.183, +0.348]
Full signal delta	115	+0.322	0.000	[+0.235, +0.409]
Cost delta, all clean pairs	115	+$0.0150	+$0.0085	[+$0.0033, +$0.0272]
Cost delta, both hard-passed	81	+$0.0210	+$0.0120	[+$0.0106, +$0.0320]
Wall delta, both hard-passed	81	+8.9 s	+3.9 s	[+4.3 s, +13.9 s]
Agent delta, both hard-passed	81	+9.0 s	+3.1 s	[+4.3 s, +13.8 s]
Repo-search delta, both hard-passed	81	−0.111	0.000	[−0.815, +0.556]
Cost delta, both signal-passed	33	+$0.0028	+$0.0025	[−$0.0086, +$0.0123]
Wall delta, both signal-passed	33	+3.9 s	+2.4 s	[+1.5 s, +6.9 s]
Repo-search delta, both signal-passed	33	+0.152	0.000	[−0.455, +0.818]

Table 12. Advantage pack: Yare vs baseline paired effects.

On the headline endpoints, hard pass per pair and signal pass per pair, Yare versus baseline clears zero (+0.261 hard, +0.322 signal). On the same-task pairwise rows where both arms succeeded, Yare is

slightly more expensive per pair (+$0.0210, CI [+$0.0106, +$0.0320]) and slightly slower per pair (+8.9 s, CI [+4.3, +13.9]). Same-task repository-search per pair is roughly flat.

The two views are not in tension. Yare attempts and completes harder work per tuple, so it spends a little more on each tuple while finishing many more tuples. Per-success spend ends up lower; per-pair spend on the same task ends up slightly higher. That is the "tasks solved per budget" shape, not "same task cheaper".

Subtrack split, three arms

Variant	Hard	Signal	Cost / hard	Agent / hard	Search / hard
baseline	19/22	9/22	$0.0725	37.2 s	4.53
Yare	22/22	21/22	$0.0686	32.2 s	3.36
Headroom	19/22	15/22	$0.0829	44.6 s	4.42

Table 13. Behaviour subtrack of the advantage pack (22 paired tuples per arm). Two seeds of one behaviour scenario are dropped for `env_probe` Headroom taints.

Variant	Hard	Signal	Cost / hard	Agent / hard	Search / hard
baseline	60/91	24/91	$0.1816	91.1 s	8.68
Yare	87/91	49/91	$0.1434	72.2 s	5.45
Headroom	60/91	21/91	$0.1858	105.4 s	9.35

Table 14. Conversation subtrack of the advantage pack (91 paired tuples per arm).

The conversation subtrack carries most of the per-success efficiency story. Yare's cost per hard pass is 21% below baseline and 23% below Headroom; agent time per hard pass is 21% below baseline and 31% below Headroom; repository-search per hard pass drops 37% below baseline and 42% below Headroom. Headroom on the conversation surface is materially worse than baseline on signal (21 vs 24) and on every other reported metric, despite spending more agent time. The behaviour subtrack reverses the Headroom-versus-baseline picture: Headroom does lift signal over baseline on front-of-turn work (15 vs 9), but Yare still leads (21 vs 15) at lower cost and shorter agent time.

Yare versus baseline paired effects on the underlying current-table view (the per-pair hard, signal, cost, and wall CIs are unchanged from the two-arm reporting because those CIs do not depend on the Headroom arm): behaviour subtrack hard delta +0.125, CI [0.000, +0.292]; behaviour subtrack signal delta +0.500, CI [+0.292, +0.708]; conversation subtrack hard delta +0.297, CI [+0.209, +0.396]; conversation subtrack signal delta +0.275, CI [+0.187, +0.363]. Both subtracks are necessary: behaviour shows the claim survives the front-of-turn surface, and conversation shows it survives the harder mid-task surface where injection fires repeatedly.

7 Headline figure: budget dominance

Yare is more expensive per turn but uses fewer turns to complete more tasks. A budget-dominance curve plots that shape directly: cost or wall-time budget on the x-axis, tasks completed within that budget on the y-axis.

At every budget on the advantage pack, Yare completes more tasks than baseline. The gap widens as the budget grows: Yare keeps converting harder seeds while baseline plateaus at 79/113 hard pass and 33/113

signal pass on the triple-clean overlap. Mean cost on a same-task pair is the wrong summary statistic here because it discards every tuple the baseline never finished, which is exactly where Yare's advantage lives.

8 Cross-run claim ladder

Each of the three layers is a stricter test than the one before. Together:

1. **Vague.** Yare answers open-ended prior-context prompts with substantially higher hard- and signal-pass rates, lower total cost, lower cost per hard pass, and lower cost per signal pass. No clean signal-loss cells across 70 paired comparisons. Pass-conditioned cost and wall deltas are negative when baseline succeeded: Yare was cheaper and faster on every seed baseline could complete, and it completed more.
2. **Behaviour.** Yare beats both baseline and Headroom on hard pass, signal pass, and signal per dollar. Wall time is lower against baseline; agent time is the fair Headroom alignment. Per-pair cost is neutral. Zero clean signal-loss cells across 223 paired comparisons.
3. **Advantage pack with Headroom.** On the purpose-built three-arm pack, Yare completes more tasks than both stock Claude Code and Headroom, at lower cost per completed task and lower agent time per completed task against both arms. Headroom does not lift hard pass over baseline and adds only a small signal lift while costing more and exploring more. Three Headroom tuples in the conversation subtrack also write native Claude memory files; Yare and baseline write none. Same-task pairwise rows show small positive cost and wall-time deltas consistent with Yare attempting and completing harder work per tuple, which is why budget dominance is the right summary, not same-task latency.

9 What the result is not

Yare does not make every coding task faster or cheaper. When baseline already succeeds on a task, Yare's same-task per-pair cost is slightly higher and same-task wall time is slightly slower (advantage pack: $+\$0.0210$, $+8.9$ s, CIs that do not cross zero). Yare does not make easy successful tasks cheaper; it makes hard tasks completable. Expecting it to beat a strong general-purpose coding agent on a self-contained refactor is the wrong test.

All three layers use `claude-sonnet-4-5`; behaviour on Opus, Haiku, or newer Sonnet revisions is untested.

10 Limitations

Open implementation items: incremental JSONL refresh on the `compose.enabled` path, scoring writeback for `USER_AUTHORED_BOOST` and citation priors, hard `PreToolUse` blocking for pinned policy, and broader action-mining coverage. On the architecture side, dense retrieval can mix up negation, time, and replaced claims; pre-turn retrieval can miss because the model has not yet reasoned about the task; several keyword lists assume English and common HTTP and POSIX wording; and on large projects most of the cost comes from offline typed-claim extraction rather than local chunk indexing. On the harness, the advantage pack runs at a single model and a single seed count, and the taint scanner does not yet cover every host-transcript leak path.

11 Further work

Petri-style memory audits. Anthropic's Petri framework is an alignment-auditing tool that deploys an automated auditor agent to test a target model through multi-turn conversations with simulated users and tools, then has a judge score the resulting transcripts across multiple dimensions. Petri itself is not a coding-memory benchmark, but the harness shape is the relevant pointer. A Yare version would seed each scenario with the memory challenge under test (stale memories, missing prior decisions,

misleading near-matches, retrieved chunks the agent should ignore), and a Petri-style auditor would drive the multi-turn coding interaction. The judge would score not just final correctness but whether the agent relied on the right memory, ignored bad memory, or wasted turns rediscovering context. That complements scripted check harnesses rather than replacing them.

Citation faithfulness. Recent RAG citation work distinguishes citation correctness from citation faithfulness: a cited source can support a claim even if the model did not rely on it. Yare can join injection logs, citation logs, file edits, and Bash calls to ask, when memory was injected and cited, whether the agent's action actually depended on it. If unfaithful citations predict failed checks, citation faithfulness becomes a useful signal before a run finishes rather than an after-the-fact paper metric.